Associative Context Memory: Persistent Experience-Based Memory for LLM Coding Agents Beyond Context Window Dependency

Hiroshi Yamato Keio University / Signal compose Inc. yamato@signalcompose.com

Preprint—Draft v0.10—2026-03-08 本ファイルは日本語翻訳版です。原文は *acm-preprint-draft.md* を参照。

概要

大規模言語モデル（LLM）ベースのコーディングエージェントは、根本的な制約を抱えている。その記憶はコンテキストウィンドウに完全に束縛されており、トークン数の増加に伴い品質が劣化する——Context Rot として知られる現象である。コンテキスト圧縮、セッション分割、検索拡張生成（RAG）といった既存の緩和策は症状に対処するものの、コンテキストウィンドウサイズへの根本的な依存を解消するものではない。本研究では、**Associative Context Memory (ACM)** を提案する。ACM は LLM コーディングエージェントのための外部永続メモリアーキテクチャであり、暗黙的なユーザーフィードバックシグナルから成功・失敗体験を捕捉し、コンテキストウィンドウ容量に依存することなくセッションをまたいだ学習を可能にする。本研究の主要な貢献は、コーディングエージェントインタラクションにおける暗黙的フィードバックシグナルの分類体系とその強度序列である。すなわち、インタラプトイベント（Ctrl+C / 強制停止）とそれに続くインタラプト後の対話、rewind 操作、修正指示、中断なしのタスク完了、という序列である。ACM を MCP（Model Context Protocol）サーバーとして設計し、既存の LLM コーディングエージェントエコシステムとの互換性を確保する。また、コンテキストウィンドウ依存の低減とセッション反復によるタスク性能の向上を評価するための実験設計を提案する。本論文は ACM をポジションペーパーおよびシステム提案として提示するものであり、主要な実装上の前提を確認する Phase 0 実現可能性プローブを含む。本格的な実験的検証は今後の研究に委ねる。

---

1. はじめに

Claude Code や OpenAI Codex に代表される LLM ベースのコーディングエージェントは、ソフトウェア開発タスクにおいて顕著な能力を示している。しかし、これらは根本的なアーキテクチャ上の制約を共有している。すなわち、すべてのコンテキスト——会話履歴、ツール出力、システムプロンプト、検索されたドキュメント——が有限のコンテキストウィンドウに収まる必要がある。セッション内でトークン数が増加するにつれ、モデルの性能は測定可能な形で劣化する。近年の実証研究はこの現象を **Context Rot** [CITE: Chroma Research 2025] として体系的に特徴づけ、トーク

ン全体に注意が分散し、コンテキストが蓄積されるにつれてタスク成功確率が低下することを示している。

この問題に対する現在のアプローチは 2 つのカテゴリに分類される。

圧縮・分割アプローチ（auto-compact、Plan Mode、手動セッションリセット）は、Context Rot をセッション管理の問題として扱う。コンテキストサイズの削減により劣化を緩和するが、セッションをまたいで知識を引き継ぐ機構を持たない。新しいセッションはすべてゼロから始まる——我々はこれを「**健忘エージェント（amnesiac agent）問題**」と呼ぶ。

検索拡張アプローチ（Serena [CITE]、RAG ベースのコンテキスト拡張）は、エージェントの関連情報へのアクセスを改善するが、既存の成果物（コード、ドキュメント）の検索に焦点を当てており、何が成功し何が失敗したかという経験的知識の捕捉は行わない。

いずれのカテゴリも、より深い問いに答えていない。**エージェントは自身のインタラクション履歴から、コンテキストウィンドウとセッションをまたいで持続する形で学習できるか？**

人間の認知は示唆に富む対照を提供する。経験豊富な開発者は過去のデバッグセッションの逐語的な記憶を保持しない。代わりに連想パターンを蓄積する。「この種の認証エラーを見たとき、うまくいったアプローチは X、失敗したアプローチは Y。」この知識は疎で、圧縮されており、生のエピソード的詳細として保存されるのではなく、連想検索の手がかりによってインデックスされている。

本研究では、LLM コーディングエージェントのための永続的な外部メモリ層である **Associative Context Memory (ACM)** を提案する。ACM は以下を実現する。

1. 暗黙的なユーザーフィードバックシグナルから成功・失敗体験を捕捉する
2. 連想キーでインデックスされた構造化・検索可能な形式で体験を保存する
3. 過度なコンテキスト消費なしに、関連する過去の体験を新しいセッションに注入する
4. 持続的な性能のためのコンテキストウィンドウサイズへの依存を低減する

本研究の中心的な新規性は、コーディングエージェントインタラクションにおいて利用可能な**暗黙的フィードバックシグナルの分類体系**である。我々は、インタラプトイベント——Ctrl+C または同等の手段によるエージェント実行の強制停止——を、特にその直後の対話と組み合わせた場合に、ユーザー不満の最も強力なシグナルとして同定する。このシグナルは PTY（擬似端末）レベルで容易に観測可能であるにもかかわらず、エージェントフィードバックに関する先行研究では見過ごされてきた。

本論文の構成は以下の通りである。第 2 節ではコンテキスト管理とエージェントメモリに関する関連研究をレビューする。第 3 節では ACM フレームワークを提示する。第 4 節では暗黙的フィードバックシグナルの分類体系を定義する。第 5 節では ACM を評価するための実験設計を記述する。第 6 節では含意と限界を議論する。第 7 節で結論を述べる。

---

2. 背景と関連研究

2.1 コンテキストウィンドウの制約と Context Rot

Transformer アーキテクチャの self-attention 機構はコンテキストウィンドウ内のすべてのトークンに対して動作し、計算量はトークン数 n に対して $O(n^2)$ でスケーリングする。計算コストに加えて、実証研究により、推論・指示追従タスクにおけるモデル性能がコンテキスト長の増加とともに劣化することが示されている。Chroma Research [CITE] はこれを「Context Rot」として体系的に特徴づけ、Claude Opus 4 や Sonnet 4 を含む最先端モデルであってもコンテキストが充填されるにつれて測定可能な性能劣化を示すことを実証した。

実務者の間では、コンテキストウィンドウの最初の約 40%（「Smart zone」と呼ばれる）の範囲内で性能が最も信頼できるという経験則が観察されており、この閾値を超えると劣化が顕著になる [CITE: Geoffrey Huntley]。

2.2 コーディングエージェントにおけるコンテキスト管理

コーディングエージェントにおける既存のコンテキスト管理戦略には以下が含まれる。

Auto-compact: コンテキストが容量に近づいた際のセッション履歴の要約。重大な弱点として、要約がコンテキスト容量の限界付近で動作する劣化したモデルによって実行される——まさに要約品質が最も低い条件である。

Plan Mode (Claude Code): 計画と実装を、クリーンなコンテキストウィンドウを持つ別個のセッションに分離する。効果的だが、手動のワークフロー規律を必要とし、クロスセッションメモリを提供しない。

CLAUDE.md と永続的な指示: セッション開始時に読み込まれる静的ドキュメント。安定したプロジェクトコンテキストには有用だが、インタラクション履歴に適応しない。

2.3 エージェントメモリシステム

LLM エージェントのための永続メモリの研究領域は急速に発展している。ACM をこの領域に位置づけるため、アプローチが異なる主要な次元を同定する。

MemGPT / Letta [CITE] は、コンテキストウィンドウを「RAM」、外部ストレージを「ディスク」として扱う階層型メモリアーキテクチャを導入し、エージェントが自律的に情報をページイン・ページアウトする。最も影響力のある汎用エージェントメモリアーキテクチャであるが、会話エージェント向けに設計されており、結果に基づく重み付けメモリ選択やコーディング固有のフィードバックシグナルには対応していない。

Mem0 および A-MEM [CITE] は、セマンティック検索とグラフベースの組織化を備えたメモリ層を

提供する。A-MEM [CITE: arXiv:2502.12110] は動的なノート構築とリンク生成を通じて柔軟性を向上させている。いずれのシステムもメモリエントリを結果の品質によって区別せず、すべてのインタラクションが成功・失敗による重み付けなしに保存・検索される。

A-MAC (Adaptive Memory Admission Control) [CITE: arXiv:2603.04549, 2026] は、ACM に構造的に最も類似した先行研究である。A-MAC はメモリ受入を 5 つの価値シグナル（将来の有用性、事実の確信度、意味的新規性、時間的新しさ、コンテンツ種別の事前確率）に分解する。しかし、A-MAC は汎用的な会話メモリに対して動作し、コーディングエージェントインタラクションからの行動シグナル（インタラプト、rewind）を受入基準として利用しない。その評価は個人的・職業的な対話ドメインを対象としており、ソフトウェア開発タスクではない。

REMEMBERER [CITE: Zhang et al., 2023] は、Q 値を含むインタラクション記録のテーブルとしてエピソディックメモリを実装し、ベース LLM のファインチューニングなしに強化学習によって更新する。これは ACM の体験スコアリングと構造的に関連するが、PTY インターフェースから直接観測可能な暗黙的行動フィードバックシグナルを使用するのではなく、強化学習による更新を適用する。

Self-Generated In-Context Examples [CITE: Sarukkai et al., NeurIPS 2025] は、成功したタスク軌跡を将来のタスクにおける in-context examples として保存し、顕著な性能向上（ALFWorld で 73%□ 89%）を実証している。これはエージェントにおける体験ベースのメモリの核心的前提を検証するものである。ACM はこの方向を以下の点で拡張する。(1) 成功とともに失敗体験を含める、(2) 二値的なタスク完了ではなく行動フィードバックに基づくシグナル強度分類体系を導入する、(3) コーディングエージェントインタラクションに特化する。

Letta Code は、MemGPT/Letta アーキテクチャに基づくメモリファーストのコーディングエージェントである。Terminal-Bench で優れた結果を達成しているが、PTY レベルの行動シグナルをメモリ品質指標として公開・活用していない。

How Memory Management Impacts LLM Agents [CITE: arXiv:2505.16067, 2025] は、LLM エージェントの「experience-following property（体験追従特性）」——エージェントが誤ったものを含むメモリからのパターンを複製する傾向——を実証的に示している。この知見は ACM における成功エントリと失敗エントリの区別を直接的に動機づける。結果に基づく重み付けメモリ選択なしでは、失敗体験はエラーを防止するのではなく伝播させることになる。

Reflexion [CITE: Shinn et al., arXiv:2303.11366, NeurIPS 2023] は、タスク試行の失敗後に言語的な自己フィードバックを生成し、それをエピソディックメモリとして保存することで後続の試行に活用する。Reflexion の構造は ACM の失敗エントリ機構と類似しており、両者とも失敗後の情報を捕捉して将来の行動を導く。重要な違いとして、Reflexion はエージェント自身の自己省察によってフィードバックを生成するのに対し、ACM はエージェントの自己評価とは独立した、ユーザーからの暗黙的行動シグナル（インタラプトイベント、rewind 操作）を捕捉する。また、Reflexion

はメモリを検索可能な外部ストアとしてセッション間で永続化しない。

ExpeL [CITE: Zhao et al., arXiv:2308.10144, NeurIPS 2023] は、成功・失敗の両方を含む過去のタスク軌跡のコレクションから、ベースモデルのファインチューニングなしに転移可能な知識を抽出し、将来のタスクにおける in-context ガイダンスとして使用する。ExPeL は ACM と「重み更新なしの体験ベース学習」という目標を共有する。ACM は回顧的な軌跡分析ではなくリアルタイムの暗黙的フィードバックシグナル（PTY レベルのインタラプト）に焦点を当てる点、および MCP ベースの既存コーディングエージェントインフラストラクチャとの統合において異なる。さらに、ExPeL の回顧的軌跡分析にはレイテンシが伴う。知識抽出はタスク完了後に行われるため、セッション中には利用できない。ACM のリアルタイムシグナル捕捉（セッション実行中に発火する hook を介して）は、将来の拡張においてセッション内の行動調整を支援し得る体験蓄積を可能にする。すなわち、セッション早期に検知された失敗シグナルが、そのセッションの残りにおけるエージェントのアプローチに反映される可能性がある。

Voyager [CITE: Wang et al., arXiv:2305.16291, 2023] は、Minecraft 環境における成功したタスク実行から永続的なスキルライブラリを構築し、セッションをまたいだ継続的学習を可能にする。Voyager はクロスセッション体験蓄積の核心的前提を検証するものである。ACM はこの方向をコーディングエージェントインタラクションに拡張し、環境報酬シグナルではなく暗黙的ユーザーフィードバックを使用して、成功とともに失敗体験を追加する。

すべての先行メモリシステムに共通する制約は、**PTY レベルの行動シグナル**——インタラプトイベント、rewind 操作、修正指示パターン——をファーストクラスのメモリ品質シグナルとして**欠いている**ことである。これらのシグナルはインタラクティブなターミナルベースのコーディングエージェントセッションにおいて一意に利用可能であり、明示的なユーザーアノテーションを必要とせず高品質な暗黙的フィードバックを提供する。

2.4 人間-LLM インタラクションにおける暗黙的フィードバック

近年の研究は、会話型 LLM インタラクションにおける暗黙的フィードバックを研究している。ICML 2025 の論文 [CITE: arXiv:2507.23158] は、WildChat および LMSYS データセットにおける暗黙的ユーザーフィードバックを分析し、フィードバックの内容（例：ユーザーが何の明確化を求めたか）が極性のみよりもモデル性能を改善することを発見した。Google での並行研究 [CITE: arXiv:2509.18361] は、Cider Chat（内部コーディングアシスタント）のテレメトリを分析し、明示的なサムズアップ/ダウンのフィードバックがインタラクションのわずか 0.6% でしか発生しないことを発見し、暗黙的行動シグナルの使用を動機づけている。

いずれの研究も PTY レベルのインタラプトイベントをフィードバックシグナルとして検討していない。Cider Chat の研究は行動アクション（強制停止、rewind）ではなく、自然言語プロンプトにおけるセンチメントに焦点を当てている。ACM のシグナル分類体系は、ターミナルベースのコーディングエージェントに特化してこのギャップを埋めるものである。

2.5 検索拡張コード生成

Serena [CITE] は、Language Server Protocol (LSP) を使用してコードベースのセマンティックインデックスを構築し、エージェントが関連するコードシンボル、定義、参照を検索できるようにする。ACM は Serena と直交する。Serena はコードベースのどの情報が利用可能かを改善し、ACM はこのコードベースでの作業に関するどの経験的パターンが利用可能かを改善する。我々の実験設計において両者の組み合わせを評価する。

2.6 ACM の位置づけ

以下の表は、最も密接に関連する先行研究と ACM の違いをまとめたものである。

システム	クロスセッション記憶	失敗体験	PTY/行動シグナル	コーディングエージェント特化
MemGPT / Letta	□	□	□	□
Mem0 / A-MEM	□	□	□	□
A-MAC	□	部分的	□	□
REMEMBERER	□	□ (Q 値)	□	□
Self-Gen ICE	□	□	□	□
Reflexion	□ (within-session only)	□	□	□
ExpeL	□	□	□	□
Voyager	□	□	□	□
Letta Code	□	□	□	□
ACM (本研究)	□	□	□	□

注*: Reflexion* は単一タスク内のリトライ間でエピソディックメモリを永続化するが、異なるタスクやコードベース間で利用可能な汎用的検索可能ストアを提供しない。*ACM* の定義では「セッション」は *Claude Code* のターミナルセッションを指し、クロスセッションメモリは異なるタスクやプ

ロジェクト間での検索を意味する。この定義に基づき、*Reflexion* をクロスセッション記憶*:* □ と分類している。

---

3. ACM フレームワーク

3.1 アーキテクチャ概要

ACM は、既存の LLM コーディングエージェントインフラストラクチャをラップする MCP（Model Context Protocol）サーバーとして設計される。この設計選択により、Claude Code、OpenAI Codex CLI、および MCP をサポートする任意のエージェントとの互換性が確保される。

```
[ACM MCP Server] — Claude Code hooks API-based implementation

  SessionStart hook
   └─ Inject ACM context (relevant past success/failure entries injected at session start)

  UserPromptSubmit hook
    └─ Capture N=3-5 turns after interrupt in real time
    └─ Detect corrective instruction text patterns (rewind fallback)

  PostToolUse hook
    └─ Record successful tool completions

  PostToolUseFailure hook
    └─ is_interrupt=true → begin failure entry generation
   └─ Note: user interrupts during tool execution are available via the official hooks API

  Stop hook
    └─ Normal completion → success entry candidate
   └─ Note: Stop hook does not fire on user interrupt (official specification)
     This "non-firing" also serves as a complementary interrupt detection signal

  SessionEnd hook
    └─ Session end → finalize and persist entries
    └─ Retrieve session log (JSONL) via transcript_path
    └─ Retrieve post-session token usage from ~/.claude.json
```

ACM 層は 3 つの機能を実行する。

1. **セッション開始時の検索**: 現在のタスクに関連するエントリについて体験 DB を照会し、圧縮された表現をセッションコンテキストに注入する
2. **セッション中のシグナル監視**: PTY インターフェースと会話から暗黙的フィードバックシグナルを捕捉する
3. **セッション終了時の体験書き込み**: 利用可能なシグナル（第 4 節で詳述）を用いてセッションをスコアリングし、成功または失敗エントリとして保存する

3.2 体験エントリ構造

ACM は 2 種類のエントリを管理する。

成功エントリ（「これをせよ」）：

```
{
  "type": "success",
  "trigger": "Authentication bug in auth.py, JWT validation failing",
  "action": "Used FileSystemWatcher to detect changes, showed diff before applying",
  "outcome": "Tests passed, no rewind or corrective instructions",
  "retrieval_keys": ["auth", "JWT", "FileSystemWatcher", "diff"],
  "signal_strength": 0.85,
  "session_id": "...",
  "timestamp": "..."
}
```

失敗エントリ（「これをするな」）：

```
{
  "type": "failure",
  "trigger": "Authentication bug in auth.py, JWT validation failing",
  "action": "Attempted bulk rewrite of all related files simultaneously",
  "outcome": "Interrupt event detected. Post-interrupt dialogue: 'not like that, just
  "retrieval_keys": ["auth", "bulk rewrite", "simultaneous changes"],
  "signal_strength": 0.95,
  "interrupt_context": {
    "turns_captured": 3,
    "dialogue_summary": "User wanted incremental diff-based changes, not bulk rewrite"
  },
  "session_id": "...",
  "timestamp": "..."
}
```

3.3 検索と注入

各セッション開始時に、ACM は以下を実行する。

1. ユーザーの初期タスク記述からキーワードを抽出する
2. 体験 DB 内の `retrieval_keys` に対してベクトル類似度検索を実行する
3. 上位 K 件の関連エントリ（成功・失敗の両方）を検索する
4. 圧縮されたサマリーをセッションコンテキストに注入する:

```
[ACM Context]
Past relevant experience:
- SUCCESS: For JWT auth bugs → use incremental diff approach (strength: 0.85)
- FAILURE: For JWT auth bugs → avoid bulk file rewrites, user interrupted (strength: 0.95)
  Note: "just change the diff, not everything"
Details available at: ~/.acm/experiences/exp_20260215_auth.json
```

注入は意図的にコンパクトである。詳細なエントリは外部に保存されパスで参照されるため、コンテキストウィンドウの膨張を回避する。これは「インデックスであってコンテンツではない」原則を実装するものである。エージェントは必要に応じて詳細をどこで見つけられるかを知っているが、無条件にロードすることはない。

3.4 体験のスコアリングと昇格

セッション終了時に、ACM は利用可能なシグナル（第 4 節で詳述）を用いてセッションをスコアリングし、昇格閾値を超えるエントリを体験 DB に書き込む。混合シグナルを持つセッションは、成功エントリ（正常に完了したサブタスク用）と失敗エントリ（インタラプトや rewind をトリガーしたサブタスク用）の両方を生成する場合がある。

---

4. 暗黙的フィードバックシグナル分類体系

本研究の中心的な貢献は、LLM コーディングエージェントインタラクションにおいて利用可能な暗黙的フィードバックシグナルの体系的分類である。明示的フィードバック（評価、サムズアップ/ダウン）とは異なり、暗黙的シグナルは行動的なものである——意図的なユーザーアクションを必要とせず、インタラクションインターフェースから観測可能である。

以下の分類体系を提案する。シグナル強度の順（最強から最弱）に記述する。

Level 1: インタラプト + インタラプト後の対話（最高強度）

シグナル: ユーザーがエージェント実行の強制停止（Ctrl+C、Esc×2、または同等のインタラプト機構）をトリガーし、その後に修正的な対話が続く。

観測元: PTY レベルの SIGINT/インタラプトイベント；後続の会話ターン（N=3〜5 を推奨）

具体的には、Claude Code の `PostToolUseFailure` hook が `is_interrupt` ブーリアンフィールドを提供し、PTY レベルの計装を必要とせずに公式 hooks API を通じてユーザー起動のインタラプトを明示的にフラグする。`Stop` hook がインタラプト発生時に発火しないことが補完的な確認シグナルとなる。

最強である理由: インタラプトイベントは最もコストの高いユーザーアクション——進行中のプロセスの能動的な停止——を表す。インタラプト後の対話は、ユーザーがなぜ停止したかを明示的に表現することが多いため、特に価値がある。「そうじゃない」「変更しすぎ」「そのアプローチは間違い」。これは説明的な内容を伴うラベル付きの負のフィードバックを構成する。

例:

```
Agent: [begins rewriting 12 files simultaneously]
User:  [Ctrl+C]
User:  "I said fix the bug, not refactor everything"
→ Failure entry: "avoid broad multi-file changes for targeted bug fix requests"
```

Level 2: Rewind 操作（高強度）

シグナル: ユーザーが rewind 機能（Claude Code における/rewind または Esc×2）を呼び出し、会話を以前の状態に復元する。

観測元: セッションイベントログ / MCP hook

高強度である理由: Rewind はエージェント出力の意図的な取り消しを表す。エージェント出力を単に無視することとは異なり、rewind はユーザーがそのインタラクションを記録から消去したいことを示す——強い暗黙的拒否である。

インタラプトとの区別: Rewind は事後的（エージェントアクション完了後）であり、インタラプトはリアルタイム（実行中）である。両者とも強い負のシグナルであるが、異なる失敗モードを捕捉する。

Level 3: 修正指示（中強度）

シグナル: ユーザーが会話内で自然言語の修正を提供する（「それは違う」「やり直して」「そういう意味じゃない」等）。

観測元: 会話テキスト；パターンマッチングまたは LLM 分類で検出可能

中強度である理由: 言語的修正はインタラプトや rewind よりもコストが低く、根本的な失敗ではなく軽微なミスアラインメントを反映している場合がある。ただし、セッション内での繰り返しの修正指示は、信頼性の高い失敗指標である。

複合シグナル: セッション内での複数の修正指示（例：3 回以上）は、rewind イベントと同様に重み付けされるべきである。

Level 4: 中断なしの完了（ベースライン / 弱い正のシグナル）

シグナル: タスクがインタラプト、rewind、修正指示なしに完了する。テストが合格する（該当する場合）。

観測元: セッション完了イベント；ツール実行の終了コード

弱い理由: 負のシグナルの不在は成功の強い確認ではない。ユーザーが最適でない出力を受け入れた可能性があるか、注意深くレビューしていない可能性がある。ただし、テスト合格と組み合わせた場合、中断なしの完了は信頼性の高い正のシグナルである。

シグナル強度サマリー

注: 以下の強度スコアは実験設計のための作業仮説として提案された仮説値である。今後の研究において実証データに基づいて較正される予定である。この序列はユーザーアクションのコストを反映している。インタラプトは進行中のプロセスの能動的な終了を必要とし（最高コスト）、rewind は意図的な状態復元を必要とし、修正指示はタイピングのみを必要とし、沈黙はゼロコストである。

シグナル	強度スコア (0〜1)	方向	捕捉方法
インタラプト + インタラプト後の対話	0.90〜1.00	負	PTY SIGINT + N ターン
Rewind	0.75〜0.90	負	セッションイベントログ
修正指示 (3 回以上)	0.60〜0.80	負	会話テキスト
修正指示 (1 回)	0.30〜0.50	負	会話テキスト
テスト合格 + 中断なし	0.70〜0.85	正	終了コード + セッションログ

シグナル	強度スコア (0〜1)	方向	捕捉方法
中断なし (テストなし)	0.40〜0.60	正	セッションログ

シグナル強度スコアは、検索時に体験エントリの重み付けに使用される。高強度のエントリが優先的に表示される。

---

5. 実験設計

5.0 Phase 0: 実現可能性プローブ

本格的な実験評価に先立ち、ACM の基盤となる主要な技術的前提が Claude Code 環境内で実現可能であるかを検証するための実現可能性プローブを実施した。

知見 1: PTY インタラプトシグナルは公式 hooks API を通じてアクセス可能である。 Claude Code の `PostToolUseFailure` hook は `is_interrupt` ブーリアンフィールドを提供し、ユーザーが Ctrl+C または同等の手段でツール実行を中断した場合に `true` に設定される。これは ACM の Level 1 シグナル（インタラプト検知）が PTY レベルの計装なしに実装可能であることを確認するものである。

知見 2: セッションログは体験エントリ生成に十分なデータを提供する。 セッションログは `~/.claude/projects/`配下に JSONL ファイルとして保存され、完全な会話履歴、ツール呼び出し、ISO 8601 タイムスタンプを含む。hook ペイロード内の `transcript_path` フィールドにより、ACM は任意の hook 呼び出し時点で現在のセッションログにアクセスできる。

知見 3: セッション後のトークン使用量は回復可能である。 セッションごとのトークン使用量はセッション終了後に `~/.claude.json` に記録され、RQ4 の事後的評価を可能にする。セッション中のリアルタイムトークン数は hooks API を通じてアクセスできない。`PreCompact` hook の発火がニアキャパシティ条件のプロキシとなる。

知見 4: Rewind イベントには専用 hook が存在しない。 Claude Code は rewind 操作のための専用 hook を公開していない。ACM は `UserPromptSubmit` イベントにおける修正指示パターンとセッショントランスクリプトにおけるメッセージ数の減少を通じて、間接的に rewind を検知する。

これらの知見は、ACM のコアアーキテクチャがエージェント自体の修正なしに Claude Code hooks API 内で実装可能であることを確認するものである。

5.1 リサーチクエスチョン

RQ1: ACM はコンテキストウィンドウ依存を低減しつつ、セッション反復によるタスク性能を維持または向上させるか？

RQ2: ACM-S（成功のみ）、ACM-F（失敗のみ）、ACM-SF（両方）のうち、どのメモリ構成が最大の性能向上をもたらすか？

RQ3: 暗黙的フィードバックシグナルの種類（インタラプト、rewind、修正指示）は、下流タスク性能で測定される体験エントリの品質と相関するか？

RQ4: ACM は人為的に制約されたコンテキストウィンドウサイズの下で持続的な性能を可能にするか？

RQ5: ACM は検索ベースのアプローチ（Serena）と相補的であり、両者の組み合わせは加算的な効果をもたらすか？

5.2 実験条件

条件	説明
Control	ベース LLM エージェント、コンテキスト管理なし
Baseline-compact	エージェント + auto-compact
Baseline-Serena	エージェント + Serena
ACM-S	エージェント + ACM（成功エントリのみ）
ACM-F	エージェント + ACM（失敗エントリのみ）
ACM-SF	エージェント + ACM（成功 + 失敗エントリ）
ACM-SF + Serena	統合アプローチ

モデル能力をアーキテクチャ効果から分離するため、すべての条件で固定のオープンウェイトモデル（例：Llama 3.1 70B または Qwen2.5-Coder 32B）をベースエージェントとして使用する。Claude Code / Codex は参照ポイントとして含めるが、統制された実験変数としては扱わない。

5.3 コンテキストウィンドウ制約条件（RQ4 用）

上記の各条件を 3 つのコンテキストウィンドウサイズで評価する。

制約	トークン予算
Full	128k（モデルデフォルト）
Half	64k

制約	トークン予算
Smart zone	50k（Full の約 40%）

ACM が縮小されたコンテキストサイズで同等の性能を可能にする場合、これはアーキテクチャ的メモリがコンテキストウィンドウ容量の代替となりうることを実証するものである。

5.4 タスクスイート

コーディングエージェントの標準ベンチマークである SWE-bench [CITE: Jimenez et al., arXiv:2310.06770, ICLR 2024] は、本評価では直接使用しない。SWE-bench のタスクはシングルセッションであり、同一コードベースに対する複数セッションにわたる繰り返しインタラクションを含まない——ACM のクロスセッションメモリが効果を発揮する条件である。本タスクスイートは Context Rot とクロスセッション改善を示すように設計されており、マルチセッションの反復実行を必要とする。SWE-bench との互換性は今後の研究の方向として位置づける。

タスクは、ベースライン条件下で Context Rot を示しつつ、明確な成功/失敗基準を提供するように設計されている。

タスク A —マルチファイルバグ修正: 現実的なコードベースにおいて 5〜10 ファイルにわたる変更を必要とする埋め込みバグを修正する。テストスイートの合格で評価する。

タスク B —仕様書からの機能追加: 自然言語の仕様書に記述された機能を実装する。機能テスト + 仕様準拠の人間レビューで評価する。

タスク C —設計原則に基づくリファクタリング: 指定された設計原則（例：依存性逆転）をコードベース全体に一貫して適用する。自動リンティング + 一貫性の人間レビューで評価する。

各タスクは条件ごとに 5 回の反復セッションで実行される。セッション 1〜2 で体験 DB を構築し、セッション 3〜5 で検索の効果を測定する。クロスセッション改善率が主要な指標である。

5.5 評価指標

指標	測定方法	主要 RQ
タスク完了率	テスト合格率 / 人間評価 (0〜1)	RQ1
インタラプト回数	セッションあたりの PTY SIGINT イベント数	RQ1, RQ3
Rewind 回数	セッションイベントログ	RQ1, RQ3
修正指示回数	会話テキスト分析	RQ1, RQ3
コンテキスト効率	完了時の使用トークン数 / タスク複雑度スコア	RQ4
クロスセッション改善	Δ 完了率、セッション 1□ 5	RQ1, RQ2

指標	測定方法	主要 RQ
シグナル-品質相関	ピアソン r: シグナル強度スコア × 下流成功率	RQ3

---

6. 議論

6.1 健忘エージェント問題

現在の LLM コーディングエージェントは、設計上、健忘的である。各セッションは過去のインタラクションの知識なしに開始される。コンテキストウィンドウ管理戦略はセッション内ではこれを緩和するが、クロスセッションの知識ギャップには対処しない。ACM は経験的知識をセッションをまたいで持続し蓄積する形式で外部化することにより、この問題に直接対処する。

これは性能指標を超えた実用的な含意を持つ。ACM を備えたエージェントはプロジェクト固有の行動パターンを発達させることができる——コードベースに何が含まれているか（Serena で検索可能）だけでなく、この特定のユーザーがこの特定のコードベースで問題にどのようにアプローチすることを好むかを知ることができる。

6.2 ファーストクラスシグナルとしてのインタラプト

インタラプトイベントは、エージェントシステム研究においてフィードバック機構として十分に活用されていない。会話 AI における同等のものは、ユーザーが生成途中でウィンドウを閉じることであろう——稀であり曖昧である。コーディングエージェントにおいて、インタラプトは一般的であり意味的に豊かである。誤ったアプローチを追求するエージェントは停止され、ユーザーの後続の説明は、将来そのアプローチを回避するために必要な負の訓練シグナルそのものである。

失敗エントリの一部としてインタラプト後 N=3〜5 ターンを捕捉することは、この対話がユーザーが提供する最も明示的なフィードバックであることが多いため、特に価値がある——苛立ちに促されて、何が問題であったかを正確に言語化するのである。

6.3 強化学習との関係

ACM のシグナル分類体系は、強化学習における報酬シグナルとの構造的類似性を持つ。インタラプトイベントは大きな負の報酬に近似し、中断なしのタスク完了は正の報酬に近似し、中間シグナルが勾配を埋める。ただし、ACM は基盤モデルに対する重み更新を行わない——コンテキストレベルで動作し、関連する過去の体験をソフトガイダンスとして注入する。この区別により、ACM はファインチューニングインフラストラクチャを必要とせず、任意の LLM でデプロイ可能となる。

6.4 制約

コールドスタート: ACM は新しいタスクタイプにおける最初のセッションでは効果を発揮しない。性能向上には蓄積された体験が必要である。

体験の陳腐化: コードベースの進化により、過去の体験が誤解を招く可能性がある。時間減衰重み付けの評価を今後の研究で計画している。

インタラプトの曖昧性: すべてのインタラプトがエージェントのアプローチへの不満を示すわけではない——外部的な中断（電話、会議）を反映するものもある。インタラプト後の対話内容に基づく曖昧性解消ヒューリスティクスが必要である。

適用範囲: 現在の設計はコーディングエージェントに特化している。他のエージェントドメイン（Web エージェント、データ分析エージェント）への一般化にはシグナル分類体系の適応が必要である。

Rewind 検知: Claude Code は rewind 操作のための専用 hook イベントを提供していない。ACM は後続の UserPromptSubmit イベントにおける修正指示パターン、またはセッショントランスクリプト（transcript_path を通じて利用可能）におけるメッセージ数の減少を監視することにより、間接的に rewind を検知する。これは実践上、rewind（Level 2）を修正指示（Level 3）と統合するが、rewind をファーストクラスイベントとして公開するエージェント環境においては概念的な区別は有効なままである。

リアルタイムコンテキスト使用量: アクティブセッション内のトークン数は Claude Code hooks API を通じてアクセスできない。ACM は RQ4 の評価にエージェント設定ファイルからのセッション後トークン数を使用し、コンテキストが容量に近づいた際に発火する PreCompact hook をニアリミット条件のプロキシとして扱う。

---

7. 結論

本研究では、暗黙的なユーザーフィードバックから成功・失敗体験を捕捉する LLM コーディングエージェントのための永続的外部メモリアーキテクチャである Associative Context Memory (ACM) を提示した。ACM の主要な貢献は以下の通りである。

1. コーディングエージェントインタラクションにおける暗黙的フィードバックシグナルの分類体系。インタラプト + インタラプト後の対話を利用可能な最強のシグナルとして同定した
2. 成功（「これをせよ」）と失敗（「これをするな」）を区別する体験エントリ構造。エージェントが正のパターンと回避行動の両方を発達させることを可能にする
3. モデル修正なしに既存の LLM コーディングエージェントエコシステムと統合する MCP 互換のアーキテクチャ設計

4. クロスセッション学習とコンテキストウィンドウ依存低減を評価するための実験設計

本研究の中心的主張——アーキテクチャ的メモリがエージェントのコンテキストウィンドウ容量への依存を低減できるということ——は、エンジニアリング実務を超えた含意を持つ。検証されれば、「健忘エージェント問題」がコンテキストウィンドウのスケーリングなしに対処可能であることを示唆し、持続的なエージェント性能への補完的な道筋を提供する。

論文草稿および関連ドキュメントは https://github.com/signalcompose/okitegami_paper で公開している。

著者貢献および AI 利用開示

知的貢献と著者責任

本論文におけるすべてのリサーチクエスチョン、中核的アイデア、アーキテクチャ概念、および知的貢献は Hiroshi Yamato に帰属する。著者は本論文に記載された内容、主張、および結論に対して全責任を負う。

研究・執筆過程における AI ツールの使用

本論文は、人間主導の AI 協働ワークフローを通じて制作された。2 つの AI ツールが異なる役割で使用された。

- **Claude.ai (claude.ai, Anthropic)** は *Strategist* として機能した：セッション間の会話コンテキストの維持、文献検索と統合の支援、査読シミュレーション、およびテキストの起草・改訂の補助を担った。Strategist はファイルへの直接書き込みやコード実行は行っていない。
- **Claude Code (Anthropic)** は *Implementer* として機能した：ファイル操作の実行、論文メタデータ取得のためのウェブ検索、草稿ドキュメントの更新、およびバージョン管理（git コミットとプッシュ）を担った。すべての操作は著者からの明示的な指示のもとで実行された。

この役割分担は、ACM 自体が自動化を提案するアーキテクチャを反映している：コンテキストを保持するエージェント（Strategist）が実行エージェント（Implementer）と連携し、両者が人間の主任研究者（PI）の指揮のもとで動作する構造である。

AI が行ったこと・行わなかったこと

AI ツールが支援した事項：文献検索と要約、散文の起草と改訂、査読者フィードバックのシミュレーション、参考文献のフォーマット、バージョン管理。

AI ツールが行わなかった事項：リサーチクエスチョンの着想、ACM アーキテクチャの提案、インタラプトをシグナルとして活用する着想の発見、論文の範囲や主張に関する意思決定、内容に対する一切の責任の負担。

AI が生成したテキストはすべて、掲載前に著者によるレビュー、編集、承認を経ている。査読シミュレーションは同じ AI ツールによって実施されたものであり、それに応じて解釈されるべきである——これは構造化された品質チェックであり、独立した専門家レビューではない。プロセスの全容は、付随リポジトリ（https://github.com/signalcompose/okitegami_paper）の `docs/session-log.md` に記録されている。

References

[CITE: Chroma Research 2025] Context Rot: How Increasing Input Tokens Impacts LLM Performance. Chroma Research, 2025. https://research.trychroma.com/context-rot

[CITE: Anthropic 2025] Effective Context Engineering for AI Agents. Anthropic Engineering Blog, 2025. https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

[CITE: MemGPT] Packer, C., et al. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560, 2023.

[CITE: Mem0] Chhikara, P., et al. Mem0: The Memory Layer for Personalized AI. arXiv:2504.19413, 2025.

[CITE: A-MEM] Xu, W., et al. A-MEM: Agentic Memory for LLM Agents. arXiv:2502.12110, 2025.

[CITE: A-MAC] Zhang, G., Jiang, W., Wang, X., Behr, A., Zhao, K., Friedman, J., Chu, X., and Anoun, A. Adaptive Memory Admission Control for LLM Agents. arXiv:2603.04549, 2026.

[CITE: REMEMBERER] Zhang, Y., et al. REMEMBERER: Equipping Large Language Models with Persistent Memory via Long-Term Interaction. arXiv:2306.07929, NeurIPS 2023.

[CITE: Self-Gen ICE] Sarukkai, V., et al. Self-Generated In-Context Examples Improve LLM Agents for Sequential Decision-Making. arXiv:2505.00234, NeurIPS 2025.

[CITE: Reflexion] Shinn, N., et al. Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv:2303.11366, NeurIPS 2023.

[CITE: ExpeL] Zhao, A., et al. ExpeL: LLM Agents Are Experiential Learners. arXiv:2308.10144, NeurIPS 2023.

[CITE: Voyager] Wang, G., et al. Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv:2305.16291, 2023.

[CITE: Memory Management] Xiong, Z., et al. How Memory Management Impacts LLM Agents: An Empirical Study of Experience-Following Behavior. arXiv:2505.16067, 2025.

[CITE: Implicit Feedback ICML] Liu, Y., Zhang, M. J. Q., and Choi, E. User Feedback in Human-LLM Dialogues: A Lens to Understand Users But Noisy as a Learning Signal. ICML 2025. arXiv:2507.23158.

[CITE: Cider Chat] Nam, D., Salawa, M., and Chandra, S. Reading Between the Lines: Scalable User Feedback via Implicit Sentiment in Developer Prompts. arXiv:2509.18361, 2025.

[CITE: Serena] Oraios AI. Serena: A Coding Agent Toolkit Providing Semantic Code Retrieval and Editing via LSP and MCP. Open-source software, 2025. https://github.com/oraios/serena

[CITE: SWE-bench] Jimenez, C. E., et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? arXiv:2310.06770, ICLR 2024.

[CITE: Letta] MemGPT open source project / Letta. https://www.letta.com

[CITE: Letta Code] Letta. Letta Code: Memory-First Coding Agent. https://www.letta.com/letta-code (accessed 2026-03-08)

[CITE: Geoffrey Huntley] Huntley, G. How to Build a Coding Agent. Blog post, 2025. https://ghuntley.com/agent/ (YouTube: https://www.youtube.com/live/fOPvAPdqgPo)

Draft v0.10 — 2026-03-08 — Feedback welcome Target venue: arXiv cs.SE / cs.AI